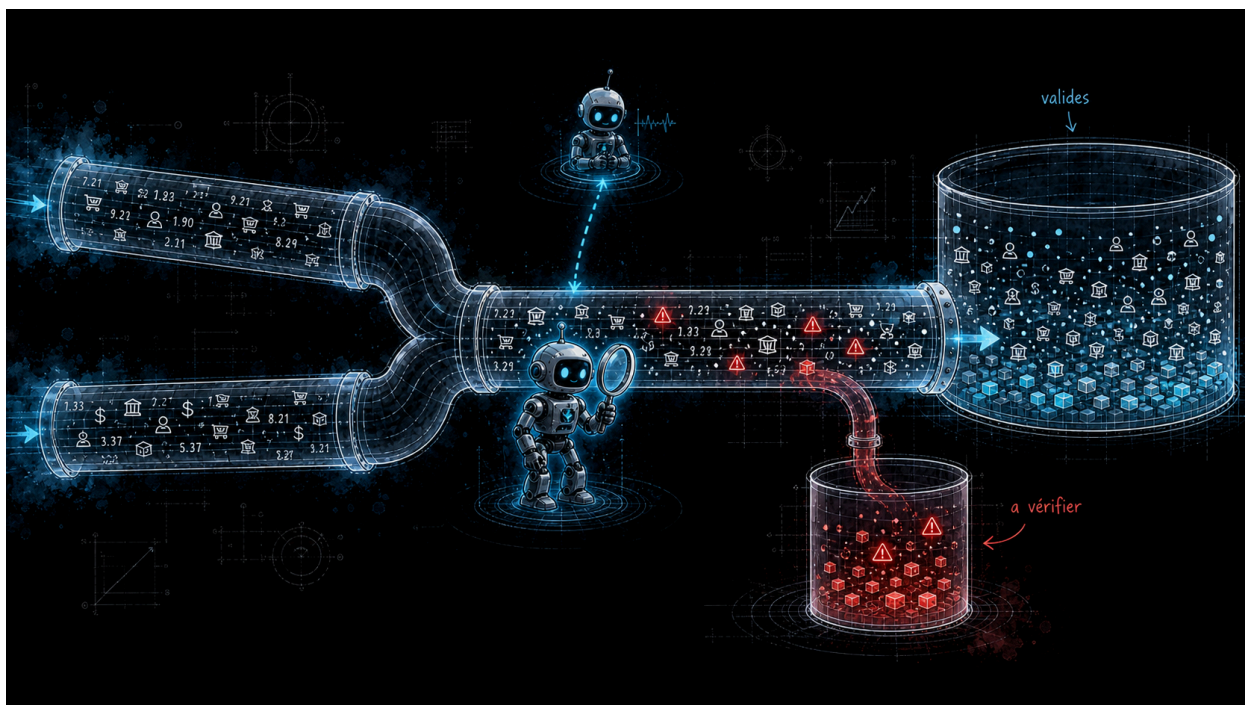
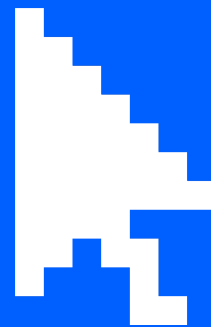


Fraud Scope

A fraud model that can't explain itself is a fraud itself.



Pipeline de détection de fraude sur des flux commercial et financier avec supervision

Introduction du sujet

Le jour où le système s'est aveuglé lui-même

Le 1^{er} août 2012, **Knight Capital Group**, l'un des plus grands teneurs de marché américains, active par erreur un algorithme de trading obsolète resté dans le code de production depuis 2005. En 45 minutes, le programme envoie des millions



d'ordres d'achat et de vente erronés sur Wall Street. Résultat : une perte de **440 millions de dollars** — soit l'intégralité des fonds propres de la société. Knight Capital est contrainte de céder ses actifs quelques jours plus tard ([NYT DealBook, 2 août 2012](#)).

Le problème ? Un **modèle non surveillé, déployé sans garde-fou**. Aucune *validation gate* avant mise en production. Aucun monitoring du comportement du système une fois actif. Aucune procédure de rollback testée.

Maintenant transposez ce scénario dans la **détection de fraude bancaire**. Votre modèle est entraîné sur les transactions de janvier à juin. En septembre, les fraudeurs changent de tactique : ils fragmentent les montants en micro-paiements et changent de marchands toutes les 48h. Votre modèle, lui, ne le sait pas. Il continue de produire des scores. Personne ne s'en aperçoit. Les faux négatifs s'accumulent silencieusement. Et quand le Directeur des Risques reçoit le rapport mensuel de chargebacks, il est déjà trop tard.

C'est le problème que vous allez résoudre.

Mise en situation professionnelle

PayTrack^(société fictive) est une fintech française spécialisée dans les paiements en ligne B2C. Elle traite 1,2 million de transactions par mois pour le compte de 3 400 marchands partenaires. Son taux de fraude est estimé à 0,17 % — soit environ 2 000 transactions frauduleuses mensuelles pour un coût de chargebacks estimé à **480 000 €/mois**.

Jusqu'ici, PayTrack détecte la fraude avec un système de **règles métier figées** : seuils de montant, blacklists de marchands connus, vérification de géolocalisation. Simple, rapide, mais aveugle à toute évolution des comportements. Résultat : 30 % de faux



positifs (clients légitimes bloqués) et 40 % de fraudes réelles manquées. Le coût indirect des faux positifs (taux d'abandon, appels au support, image de marque) dépasse désormais celui des fraudes elles-mêmes.

Le Chief Data Officer vient de vous mandater en tant qu'équipe Data Engineering & ML. Sa demande tient en **quatre questions** :

1. « Votre modèle fait-il **mieux que nos règles actuelles** ? »
2. « Comment saurai-je qu'il **se dégrade dans 3 mois** ? »
3. « Comment un analyste peut-il **comprendre pourquoi une transaction est bloquée**? »
4. « Comment déploie-t-on **une nouvelle version sans casser la production** ? »

Votre mission : construire un **pipeline ML industrialisé, explicable et surveillé** qui répond à ces quatre questions — et convaincre le CDO lors de la soutenance finale.

Les données

Vous travaillez avec le dataset public **IEEE-CIS Fraud Detection** fourni par Vesta Corporation. Il est disponible sur Kaggle :

kaggle.com/competitions/ieee-fraud-detection/data.

Caractéristique	Valeur	Remarque pédagogique
Transactions	590 540	Volume réaliste pour un POC fintech
Features	433	Identité, carte, device, historique, comportement



Taux de fraude 3,5 % Déséquilibre 1:28 — piège classique de l'accuracy

Split temporel `train.csv`
`/ test.csv` Ordre chronologique strict à respecter

⚠ Piège classique à désamorcer

Ne mélangez jamais les données passées et futures dans votre split d'entraînement. La fraude a une dimension temporelle forte : un modèle qui « voit » des données futures dans son entraînement est un modèle qui ment. Utilisez `TimeSeriesSplit`, jamais `KFold`.

Dataset complémentaire pour le challenge GNN : **Elliptic Bitcoin Dataset**, directement importable via PyTorch Geometric : [documentation EllipticBitcoinDataset](#).

Il représente 203 769 nœuds (transactions Bitcoin) reliés par 234 355 arêtes (flux de cryptomonnaies), avec 2 % de transactions frauduleuses étiquetées.

Planning indicatif — 5 jours

Jour	Thème	Livrable de fin de journée
1	EDA + illusion de l'accuracy + feature engineering temporel	<code>01_exploration.ipynb</code> + baseline métriques



2	Stratégies de resampling + XGBoost comparatif	Table comparative AUPRC/F1/temps par stratégie
3	Graph features (NetworkX) + GNN (GCN vs GAT sur Elliptic)	<code>02_modelling.ipynb</code> + analyse comparative
4	MLflow tracking + registry + serving + SHAP	<code>03_mlops.ipynb</code> + endpoint REST fonctionnel
5	Monitoring drift (Evidently)	Rapports HTML Evidently

Les tâches

L'illusion de l'accuracy

Téléchargez et explorez les données IEEE-CIS. Avant toute modélisation, construisez votre compréhension des données et démontrez pourquoi la métrique la plus naturelle est ici la plus trompeuse.

1. Réalisez une **analyse exploratoire rigoureuse** : distribution du déséquilibre de classes, patterns temporels (heure, jour, semaine), distributions des montants par catégorie (fraude vs légitime), corrélations remarquables entre features et cible.
2. Entraînez un premier modèle naïf — Régression Logistique, sans aucun prétraitement du déséquilibre. Calculez son **accuracy**.



3. Calculez ensuite son **Recall sur la classe fraude** et son **AUPRC** (Area Under the Precision-Recall Curve). Que constatez-vous ? Pourquoi l'accuracy est-elle une métrique trompeuse sur ce dataset ?

💡 **Compétence clé** : comprendre pourquoi les métriques standard échouent sur les données déséquilibrées, et choisir les métriques adaptées au contexte métier (coût d'un faux négatif >> coût d'un faux positif dans un système anti-fraude).

Feature engineering temporel

Les variables les plus discriminantes en fraude ne sont pas les features brutes, mais les **écarts par rapport au comportement habituel de l'utilisateur**.

1. Construisez des **features de vélocité** : nombre de transactions dans les 1h, 24h et 7 jours précédents pour le même client ; montant cumulé sur ces fenêtres temporelles.
2. Construisez des **features de déviation comportementale** : ratio `montant_actuel / montant_moyen_7j`, indicateur booléen « nouveau marchand », indicateur booléen « nouvel appareil ».
3. Construisez votre pipeline sklearn en respectant strictement l'**ordre temporel** (utilisez `TimeSeriesSplit`, pas de `KFold`). Pourquoi ce choix est-il impératif ici ? Quel serait le biais introduit par un `KFold` classique dans ce contexte ?

Comparaison des stratégies de resampling

Le déséquilibre de classes est un problème de fond en détection de fraude. Plusieurs stratégies existent pour y répondre. Comparez-les rigoureusement sur des métriques métier pertinentes.



Stratégie	Implémentation	Librairie
Aucune (baseline)	<code>XGBClassifier()</code> brut	xgboost
Pondération des classes	<code>scale_pos_weight</code> dans XGBoost	xgboost
Random Undersampling	<code>RandomUnderSampler</code>	imbalanced-learn
SMOTE	<code>SMOTE</code> appliqué uniquement sur train	imbalanced-learn
SMOTE + ENN	<code>SMOTEENN</code> — SMOTE + nettoyage des frontières	imbalanced-learn

Pour chaque stratégie, calculez et consignez dans un tableau : **AUPRC, Recall@seuil 0,5, F1-score, temps d'entraînement**.

⚠ Piège à éviter : data leakage par SMOTE

Appliquer SMOTE *avant* le split train/test est une erreur courante qui produit un optimisme artificiel (les exemples synthétiques générés à partir du test se retrouvent dans le train). Montrez dans votre notebook que vous l'avez évité — et expliquez pourquoi c'est important.



Quelle stratégie recommandez-vous au CDO de PayTrack ? Argumentez en termes métier (coût des faux négatifs, risque de surcharge des analystes par les faux positifs, maintenabilité en production).

Au-delà du tabulaire — Graph Neural Networks

Certains patterns de fraude sont **invisibles aux modèles tabulaires** : un fraudeur peut avoir des features individuelles parfaitement normales, mais être connecté à un réseau de comptes suspects. C'est la fraude en anneau (ring fraud) — seule l'analyse relationnelle peut la détecter.

1. Sur un sous-ensemble de 10 000 transactions IEEE-CIS, construisez un **graphe de transactions** avec `NetworkX` : nœuds = comptes clients + marchands, arêtes = transactions. Visualisez ce graphe. Identifiez visuellement au moins un cluster suspect.
2. Extrayez des **features graphe** : degré du nœud compte, nombre de marchands distincts sur 7 jours, centralité betweenness. Intégrez ces features dans votre meilleur modèle XGBoost (tâche 3). Observez et commentez le gain d'AUPRC.
3. Sur le dataset **Elliptic** (importable via `torch_geometric.datasets.EllipticBitcoinDataset`), entraînez :
 - Un **Graph Convolutional Network (GCN)** à 2 couches
 - Un **Graph Attention Network (GAT)** à 2 couches
4. Comparez leurs performances (Recall, AUPRC, temps d'inférence). Pourquoi GAT est-il potentiellement supérieur à GCN sur des données de fraude ?
5. Rédigez une **analyse comparative** des approches : ML tabulaire / ML tabulaire + features graphe / GNN pur. Quand choisiriez-vous chaque approche en production ? Prenez en compte la performance, la latence, l'interprétabilité et la maintenabilité.



Industrialisation avec MLflow

Un bon modèle non tracé est un modèle perdu. Un bon modèle non versionné est un modèle indéploable. Cette tâche vous fait parcourir le cycle complet de vie d'un modèle ML en production.

1. Mettez en place un **serveur MLflow local** ([docs MLflow](#)) : `mlflow server --host 127.0.0.1 --port 8080`. Créez une expérience `fraud-detection-paytrack`.
2. Pour chacune de vos 5 stratégies de resampling (tâche 3), loguez dans MLflow : les hyperparamètres, les métriques (AUPRC, Recall, F1, temps), et les artefacts (courbe Precision-Recall, matrice de confusion, graphe SHAP beeswarm).
3. Dans l'UI MLflow, comparez vos runs et sélectionnez le modèle candidat à la production selon : **AUPRC > 0,85 ET temps d'inférence < 50 ms**.
4. Enregistrez ce modèle dans le **MLflow Model Registry** en version `v1`, statut `Staging`. Définissez une **validation gate automatique** : si AUPRC sur holdout temporel > seuil configuré, promouvez automatiquement le modèle en `Production`.
5. (optionnel) Entraînez une version `v2` enrichie des features graphe (tâche 4b). Comparez `v1` vs `v2` dans le registry. Que recommandez-vous au CDO ?
6. Déployez le modèle de production via `mlflow models serve`. Testez l'endpoint `POST /invocations` avec une transaction JSON. La réponse doit inclure le **score de fraude** et les **top-3 SHAP values** expliquant la décision.

SHAP — expliquer chaque décision

Le CDO a posé la question : « Comment un analyste peut-il comprendre pourquoi une transaction est bloquée ? ». La réponse est SHAP (SHapley Additive exPlanations). Documentation : shap.readthedocs.io.



1. Produisez un **beeswarm plot SHAP** sur votre modèle de production : quelles sont les 10 features qui contribuent le plus aux prédictions ? Sont-ce les features que vous attendiez au regard de la logique métier ?
2. Analysez deux cas concrets :
 - Une transaction **correctement détectée comme fraude** : quelles features ont déclenché l'alerte ? L'explication est-elle cohérente avec votre connaissance métier ?
 - Une transaction **faux positif** (légitime bloquée) : pourquoi le modèle s'est-il trompé ? Quelles features l'ont induit en erreur ?
3. Un analyste fraude doit pouvoir lire l'explication sans connaître le modèle. Rédigez un **template d'explication en langage naturel** automatiquement généré à partir des SHAP values. Exemple de sortie attendue :

« Cette transaction a été bloquée principalement parce que le montant dépasse de 340 % la moyenne habituelle du client sur 7 jours, et parce qu'il s'agit d'un marchand jamais vu dans l'historique du compte. »

Monitoring de drift en production

Votre modèle est en production. Les fraudeurs évoluent. Comment le savoir avant que les chargebacks ne s'accumulent ? Cette tâche répond à la deuxième question du CDO.

1. Découpez le dataset en 3 périodes temporelles : **T0** (entraînement), **T1** (production stable), **T2** (post-évolution des patterns — modifiez artificiellement T2 en augmentant la proportion de micro-transactions frauduleuses de faible montant).
2. Avec [Evidently AI](#), générez :
 - Un **Data Drift Report** (T1 vs T2) : quelles features ont le plus dérivé ? (tests KS, PSI)



- Un **Model Performance Report** : comparez l'AUPRC sur T1 et T2. De combien le modèle s'est-il dégradé ?
 - Un **Prediction Drift Report** : la distribution des scores de fraude a-t-elle changé ?
3. Définissez des **seuils d'alerte de retraining** : si AUPRC chute de plus de X % OU si PSI > 0,2 sur une feature critique, déclenchez une alerte. Justifiez vos seuils par rapport aux contraintes métier de PayTrack.
 4. Dans un tableau synthétique, répondez à la question : « À quelle date aurait-on dû retrainner le modèle si on l'avait déployé en production sur le dataset PayTrack ? »

(Optionnel) Recherche de transactions similaires

Implémentez un moteur de **recherche de similarité** pour les analystes fraude : étant donné une transaction suspecte, retrouver dans l'historique les 5 transactions les plus similaires (légitimes ou frauduleuses) pour reconstituer un pattern.

1. Encodez vos transactions en vecteurs denses (embeddings) via votre encodeur MLP ou les SHAP values comme vecteur de features.
2. Indexez ces vecteurs dans **Qdrant** (Docker local).
3. Testez la requête : « Quelles transactions passées ressemblent le plus à cette fraude détectée ? ». L'analyste peut-il reconstituer un pattern à partir des résultats ?

Compétences visées



Bloc	Compétence	Tâche(s)
BC05	Analyser une problématique big data déséquilibrée ; concevoir et évaluer des modèles ML/DL (XGBoost, GCN, GAT) sur données tabulaires et relationnelles	1, 2, 3, 4
BC05	Construire un pipeline MLOps complet : tracking, registry, validation gate, serving REST (MLflow)	5
BC05	Appliquer des techniques d'explicabilité (SHAP) pour produire des explications opérationnelles	6
BC05	Détecter la dérive des données et des modèles en production ; définir des seuils d'alerte de retraining (Evidently AI)	7
BC04	Analyser les risques de biais, de dérive et d'attaque adversariale sur un système de détection de fraude	3, 6, 7
BC02	Piloter un projet technique en équipe ; adapter la communication aux parties prenantes (CDO, analyste, responsable technique)	Soutenance



BC01

Comparer des architectures de détection (règles vs ML vs GNN ; 4d, Soutenance batch vs serving en ligne) ; positionner le système dans la stratégie SI

Rendu

Votre soutenance : simulation d'une réunion de restitution au CDO de PayTrack.

Votre présentation doit répondre explicitement aux 4 questions posées dans le contexte. Le jury joue des rôles distincts : CDO, analyste fraude, responsable infrastructure. Préparez-vous à des questions dans les trois registres.

Structure recommandée (20 min) :

- contexte et problématique
- choix de métriques et EDA
- comparaison des modèles
- pipeline MLOps & serving
- monitoring drift
- recommandation au CDO

Repository GitHub public — fraud-scope

Le repository doit contenir les éléments suivants :



- **Notebooks Python** (maximum 3, propres, commentés, de qualité publiable) :
 - `01_exploration.ipynb` — EDA, métriques, piège de l'accuracy, feature engineering
 - `02_modelling.ipynb` — resampling, GNN, comparaison des approches
 - `03_mlops.ipynb` — MLflow tracking, registry, serving, SHAP, monitoring Evidently
- `README.md` comprenant : contexte PayTrack, données, choix algorithmiques et MLOps justifiés, résultats clés, recommandation au CDO.
- `mlruns/` — répertoire MLflow avec les runs loggés (ou lien vers serveur MLflow partagé).
- `evidently_reports/` — rapports HTML (data drift, model performance, prediction drift).
- `requirements.txt` + instructions de reproduction dans le README.

Base de connaissances

Dataset principal

IEEE-CIS Fraud Detection (Kaggle)

590 540 transactions, 433 features — Vesta Corporation

Dataset GNN

Elliptic Bitcoin — PyTorch Geometric

203 769 nœuds, 234 355 arêtes

MLOps — Tracking & Registry



[MLflow Model Registry \(docs officielles\)](#)

[Tutoriel complet MLflow](#)

Monitoring de drift

[Evidently AI — Documentation officielle](#)

[Data Drift, Model Performance, Prediction Drift](#)

Explicabilité

[SHAP — Documentation officielle](#)

[TreeExplainer, beeswarm, waterfall plots](#)

Resampling

[imbalanced-learn \(SMOTE, SMOTEENN\)](#)

[SMOTEENN — référence API](#)

PyTorch Geometric

[Documentation PyG \(GCN, GAT\)](#)

[Guide d'installation PyG](#)

NetworkX

[NetworkX — Documentation officielle](#)

[Graphes, centralité, visualisation](#)

Article de référence

[Dal Pozzolo et al. \(2019\) — Imbalanced Classification for Credit Card Fraud Detection](#)

[IEEE Transactions on Neural Networks \(TNNLS\)](#)

Article de référence GNN



Weber et al. (2019) — Anti-Money Laundering in Bitcoin: GCN for Financial Forensics

Dataset Elliptic original

Contexte historique

NYT DealBook — Knight Capital, \$440M en 45 min (2012)

Éthique & biais algorithmique

ProPublica — Machine Bias (COMPAS, 2016)